

Software Architecture Laboratory

Laboratory No. 3

Synchronization & Deadlocks

Immortals Simulation with Java 21

Students:

Andersson David Sánchez Méndez

Cristian Santiago Pedraza Rodríguez

Elizabeth Correa Suárez

Juan Sebastian Ortega Muñoz

Instructor: Professor Javier Ivan Toquica Barrera

Course: Software Architecture (ARSW)

Institution: Escuela Colombiana de Ingeniería Julio Garavito

February 13, 2026

Contents

1	Objective	3
2	Problem Description	3
2.1	Application Architecture	3
3	Theoretical Framework	3
3.1	Busy-Wait vs Wait/Notify	3
3.2	Deadlock Conditions	3
3.3	Deadlock Prevention Strategies	3
4	Part I: Producer/Consumer	4
4.1	Problem: High CPU Consumption	4
4.2	Evidence of High CPU Usage	4
4.3	Solution: BoundedBuffer with Monitors	4
4.4	Evidence of Efficient CPU Usage	4
4.5	Fast Producer / Slow Consumer	4
5	Part II: Distributed Search	5
5.1	Early Stop Implementation	5
5.2	AtomicInteger for Race-Free Counting	5
5.3	Early Stop Check	5
5.4	Evidence of Early Stop	5
6	Part III: Immortals Simulation	5
6.1	Simulation Overview	5
6.2	Invariant Calculation	5
6.3	Pause & Check Validation	6
6.4	Multiple Pause Validation	6
7	Deadlock Analysis	6
7.1	Critical Sections Identified	6
7.2	Naive Strategy (Causes Deadlock)	6
7.3	Ordered Strategy (No Deadlock)	7
7.4	TryLock with Exponential Backoff	7
7.5	Strategy Comparison	7
8	High N Validation	7
8.1	N=100 Results	7
8.2	N=1000 Results	8
8.3	N=10000 Results	8
8.4	Validation Summary	8
9	Removing Dead Immortals	8
9.1	Problem: Race Conditions with ArrayList	8
9.2	Solution: CopyOnWriteArrayList	8
10	Orderly Shutdown	9
10.1	Graceful Shutdown Pattern	9
11	Methodology	9
11.1	Test Environment	9
11.2	Execution Commands	9
11.3	Running Tests	9
12	Conclusions	9
12.1	Lessons Learned	9
13	References	9

1 Objective

This laboratory focuses on practicing minimal synchronization, avoiding deadlocks, and implementing cooperative suspension in an “immortals”-type thread system using Java 21. The simulation involves multiple concurrent threads representing immortal warriors that fight each other, requiring proper coordination to ensure correct behavior and data consistency.

The specific goals include:

- **Producer/Consumer:** Implement efficient CPU usage with `wait()/notify()` instead of busy-wait.
- **Distributed Search:** Implement early termination and atomic counters without race conditions.
- **Deadlock Prevention:** Identify and fix deadlocks using total order or `tryLock(timeout)`.
- **Pause/Resume Control:** Implement stable pause mechanism with synchronization barriers.
- **Thread-safe Collections:** Handle concurrent modifications without blocking the simulation.
- **Orderly Shutdown:** Implement graceful termination of all threads.

2 Problem Description

The immortals simulator presents several concurrency challenges across three main parts:

Deadlock Warning

1. **Part I - Busy-Wait:** High CPU consumption in producer/consumer due to spin-wait pattern.
2. **Part II - Race Conditions:** Distributed search with shared counter may have race conditions.
3. **Part III - Deadlocks:** Multiple immortals acquiring nested locks may cause circular wait.

2.1 Application Architecture

The application follows a layered architecture:

Package	Responsibility
app	Bootstrap (Main)
highlandersim	Swing UI
immortals	Domain model
concurrency	PauseController
demos	Theoretical demos
core	Bank transfers

Table 1: Application package structure

3 Theoretical Framework

3.1 Busy-Wait vs Wait/Notify

Busy-wait (spin-wait) is a pattern where a thread continuously checks a condition in a tight loop:

Listing 1: Busy-Wait Pattern (Inefficient)

```
1 while (true) {
2     if (queue.size() < capacity) {
3         queue.add(item);
4         return;
5     }
6     Thread.onSpinWait(); // CPU spinning!
7 }
```

The `wait()/notify()` pattern releases CPU when waiting:

Listing 2: Wait/Notify Pattern (Efficient)

```
1 synchronized (this) {
2     while (queue.size() == capacity) {
3         this.wait(); // Release CPU
4     }
5     queue.add(item);
6     this.notifyAll();
7 }
```

Wait/Notify Semantics

- `wait()`: Releases lock, enters WAITING state
- `notify()`: Wakes one waiting thread
- `notifyAll()`: Wakes all waiting threads
- Use `while` loop for spurious wakeups

3.2 Deadlock Conditions

A **deadlock** occurs when four conditions hold simultaneously:

1. **Mutual Exclusion:** Resources cannot be shared.
2. **Hold and Wait:** Thread holds one resource while waiting for another.
3. **No Preemption:** Resources cannot be forcibly taken.
4. **Circular Wait:** Threads form a circular chain of waiting.

3.3 Deadlock Prevention Strategies

Strategy 1: Total Order

Listing 3: Total Order Lock Acquisition

```
1 Immortal first = name.compareTo(
2     other.name) < 0 ? this : other;
3 Immortal second = name.compareTo(
4     other.name) < 0 ? other : this;
5 synchronized (first) {
6     synchronized (second) {
7         // Critical section
8     }
9 }
```

Strategy 2: TryLock with Timeout

Listing 4: TryLock with Backoff

```
1 if (this.lock.tryLock(10, MILLISECONDS)) {
2     try {
3         if (other.lock.tryLock(10, MILLISECONDS)) {
4             try {
5                 // Critical section
6             } finally {
7                 other.lock.unlock();
8             }
9         }
10    } finally {
11        this.lock.unlock();
12    }
13 }
14 // Exponential backoff on failure
```

4 Part I: Producer/Consumer

4.1 Problem: High CPU Consumption

The `BusySpinQueue` class implements a busy-wait pattern that causes excessive CPU usage:

Listing 5: `BusySpinQueue` - High CPU

```
1 // take method - CPU spinning
2 while (true) {
3     T v = q.pollFirst();
4     if (v != null) return v;
5     Thread.onSpinWait(); // Keeps CPU busy
6 }
```

4.2 Evidence of High CPU Usage

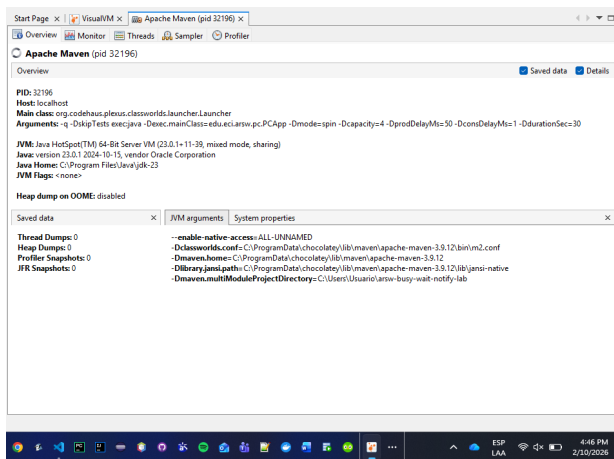


Figure 1: jVisualVM Monitor: High CPU usage with busy-wait (spin mode).

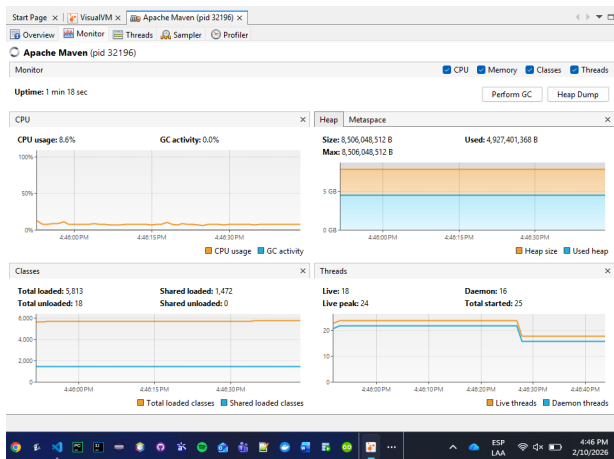


Figure 2: jVisualVM Threads: Threads constantly in `Runnable` state.

4.3 Solution: `BoundedBuffer` with Monitors

Listing 6: `BoundedBuffer` - Efficient CPU

```
1 public T take() throws InterruptedException {
2     synchronized (this) {
3         while (q.isEmpty()) {
4             this.wait(); // Release CPU
5         }
6         T v = q.removeFirst();
7         this.notifyAll();
8     }
9 }
```

```
8     return v;
9 }
10 }
```

Listing 7: `BoundedBuffer` - Put Method

```
1 public void put(T item)
2     throws InterruptedException {
3     synchronized (this) {
4         while (q.size() == capacity) {
5             this.wait(); // Wait when full
6         }
7         q.addLast(item);
8         this.notifyAll();
9     }
10 }
```

4.4 Evidence of Efficient CPU Usage

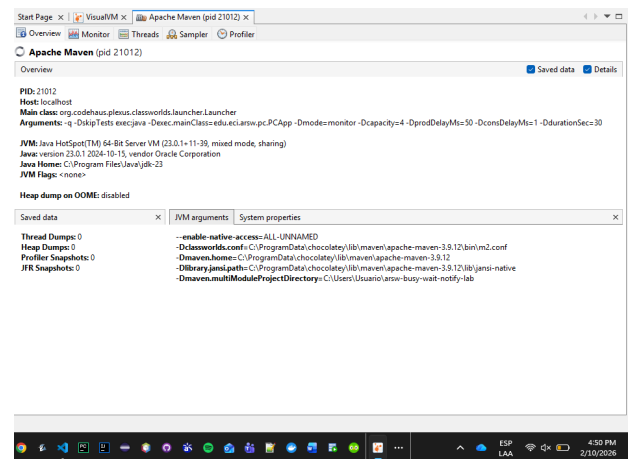


Figure 3: jVisualVM Monitor: Low CPU usage with monitors.

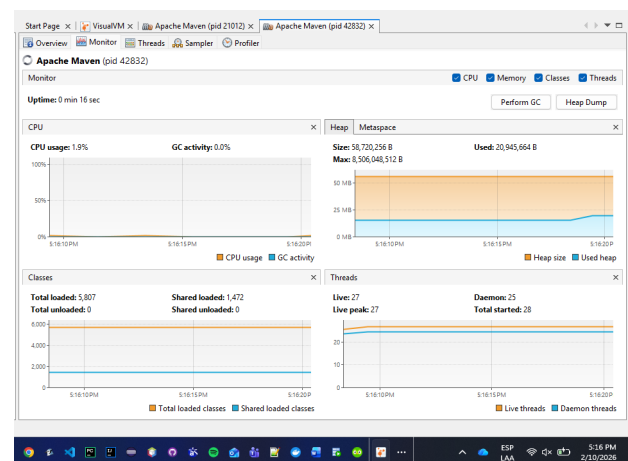


Figure 4: jVisualVM Threads: Threads in `Waiting` state when idle.

4.5 Fast Producer / Slow Consumer

When the producer is faster than the consumer with a bounded queue:

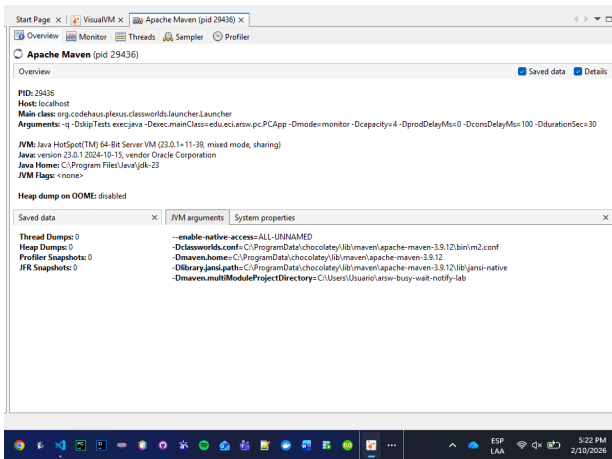


Figure 5: Producer waits efficiently when queue is full.

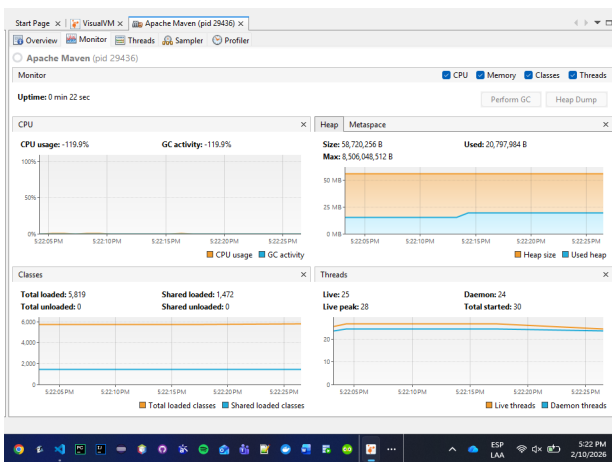


Figure 6: Producer in WAITING state when queue reaches capacity.

5 Part II: Distributed Search

5.1 Early Stop Implementation

The distributed search pattern was adapted to the greyhound race domain:

Concept	Dogs Race
Search threads	Galgo threads
Occurrence counter	AtomicInteger
ALARM_COUNT	arrivalAlarmCount

Table 2: Mapping to distributed search pattern

5.2 AtomicInteger for Race-Free Counting

Listing 8: Atomic Counter - No Race Condition

```

1 // BEFORE: Race condition possible
2 final int position = nextPosition++;
3
4 // AFTER: Atomic CAS operation
5 final int position =
6     nextPosition.getAndIncrement();

```

5.3 Early Stop Check

Listing 9: Early Stop in Thread Loop

```

1 while (paso < carril.size()) {
2     control.awaitIfPaused();
3
4     // Early-stop check
5     if (registry.hasReachedThreshold()) {
6         stoppedEarly = true;
7         break; // Don't traverse remaining
8     }
9     // ... continue stepping
10 }

```

5.4 Evidence of Early Stop

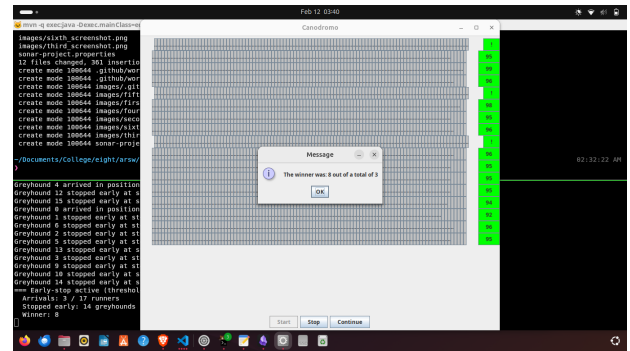


Figure 7: With threshold=3: Race stops after 3 arrivals.

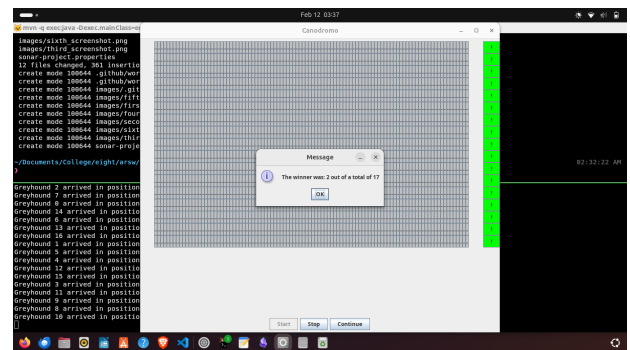


Figure 8: Without threshold: All 17 greyhounds finish.

6 Part III: Immortals Simulation

6.1 Simulation Overview

The simulation models N immortals running concurrently. Each immortal is represented by one thread and repeatedly attacks another immortal:

- N : number of immortals ($-Dcount$)
- H : initial health ($-Dhealth$)
- M : damage per hit ($-Ddamage$)

Fight rule:

- Attacker decreases opponent health by M
- Attacker increases own health by $M/2$

6.2 Invariant Calculation

Initial total health:

$$S_0 = N \cdot H = 8 \cdot 100 = 800$$

Change per fight:

$$\Delta = -M + \frac{M}{2} = -10 + 5 = -5$$

After k fights:

$$S(k) = S_0 + k\Delta = 800 - 5k$$

Important Note

The constant invariant ($N \times H$) is **not** maintained because each fight reduces the global total by $M/2$. However, the implementation-based invariant $S(k) = S_0 - 5k$ is maintained.

6.3 Pause & Check Validation

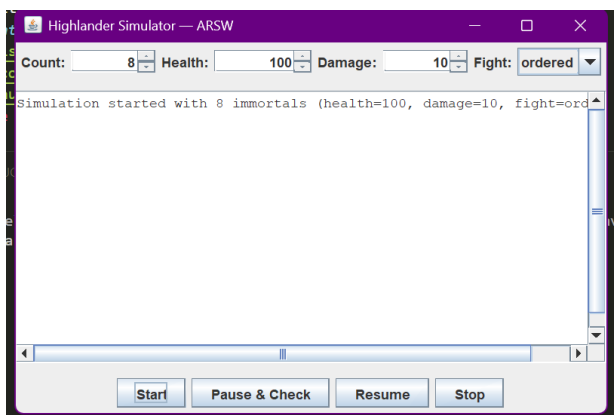


Figure 9: Pause & Check showing Score=146, Total Health=70.

Using the formula: $S(146) = 800 - 5(146) = 70$ — matches observed value!

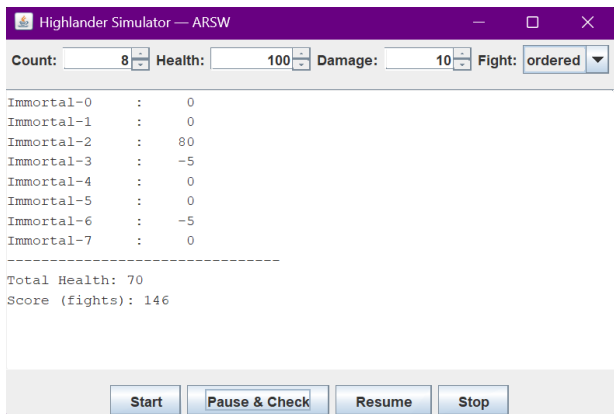


Figure 10: Detailed health breakdown per immortal.

6.4 Multiple Pause Validation

Pause	Score	Observed	Expected
1	1712	2864	$8000 - 3(1712)$
2	2575	275	$8000 - 3(2575)$
3	2648	56	$8000 - 3(2648)$

Table 3: Multiple pause validation (all match)

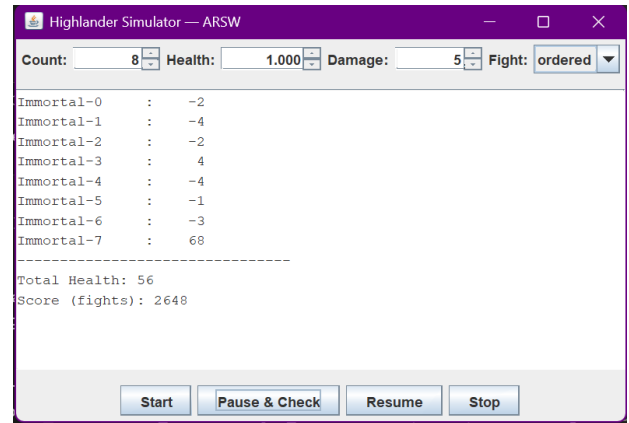


Figure 11: Pause test with k=2648, observed total=56.

7 Deadlock Analysis

7.1 Critical Sections Identified

The main critical section is the fight operation:

Listing 10: Critical Section - Fight Operation

```
1 other.health -= this.damage; // Write
2 this.health += this.damage / 2; // Write
3 scoreBoard.recordFight(); // Update
```

7.2 Naive Strategy (Causes Deadlock)

Listing 11: Naive Lock Acquisition

```
1 synchronized (this) {
2     synchronized (other) {
3         // Fight logic
4     }
5 }
```

The deadlock scenario:

Thread A (5 vs 3): Thread B (3 vs 5):
 Lock(5) OK Lock(3) OK
 wants(3) BLOCKED wants(5) BLOCKED
 --> DEADLOCK <--

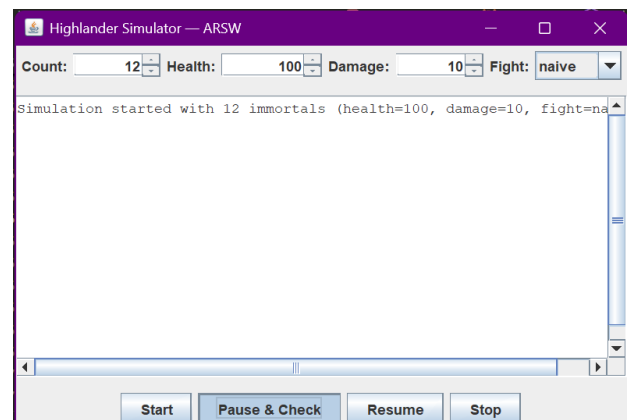


Figure 12: UI frozen with naive mode after a few seconds.

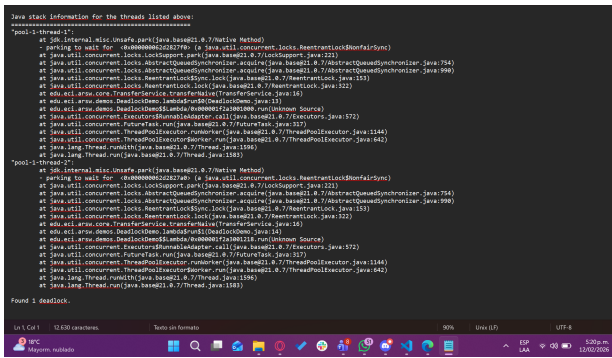


Figure 13: jstack output confirms deadlock.

7.3 Ordered Strategy (No Deadlock)

Listing 12: Ordered Lock Acquisition

```

1  immortal first = this.name.compareTo(
2      other.name) < 0 ? this : other;
3  immortal second = this.name.compareTo(
4      other.name) < 0 ? other : this;
5  synchronized (first) {
6      synchronized (second) {
7          // Fight logic
8      }
9  }

```

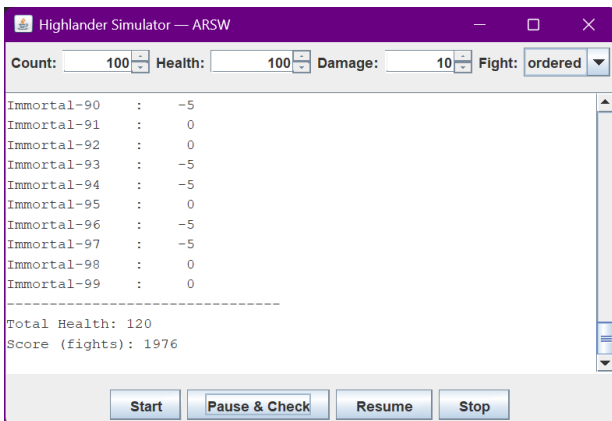


Figure 14: UI remains responsive with ordered mode.

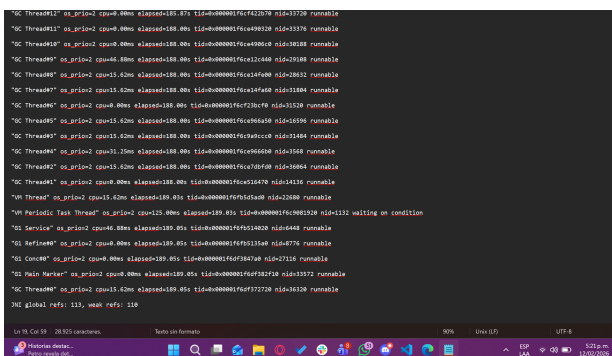


Figure 15: jstack shows no deadlock with ordered mode.

7.4 TryLock with Exponential Backoff

Listing 13: TryLock Strategy

```

1  for (int attempt = 0;
2      attempt < MAX_RETRIES; attempt++) {

```

```

3      if (this.lock.tryLock(10, MILLISECONDS)) {
4          try {
5              if (other.lock.tryLock(10, MILLISECONDS)) {
6                  try {
7                      // Fight logic
8                      return; // Success
9                  } finally { other.lock.unlock(); }
10             }
11             } finally { this.lock.unlock(); }
12         }
13         // Exponential backoff with jitter
14         Thread.sleep(backoff + jitter);
15         backoff = Math.min(backoff * 2, MAX_BACKOFF);
16     }

```

7.5 Strategy Comparison

Aspect	Naive	Ordered	TryLock
Lock mechanism	synch	synch	ReentrantLock
Lock order	this→other	alphabetical	any
Deadlock?	YES	NO	NO
Livelock?	NO	NO	NO
Blocking	infinite	infinite	timeout

Table 4: Fight strategy comparison

8 High N Validation

8.1 N=100 Results

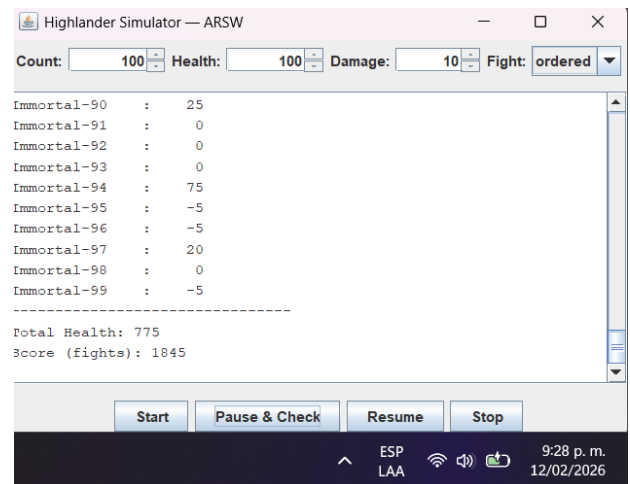


Figure 16: Validation N=100 with ordered strategy.

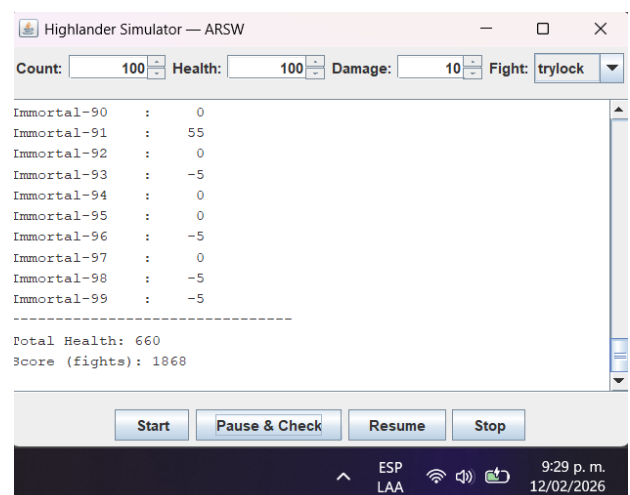


Figure 17: Validation N=100 with trylock strategy.

8.2 N=1000 Results

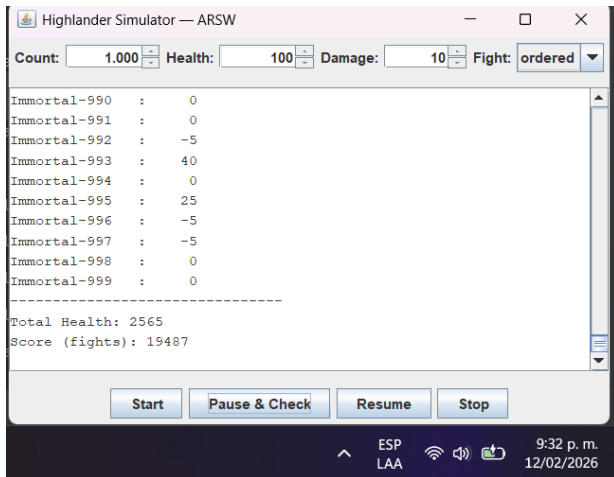


Figure 18: Validation N=1000 with ordered strategy.

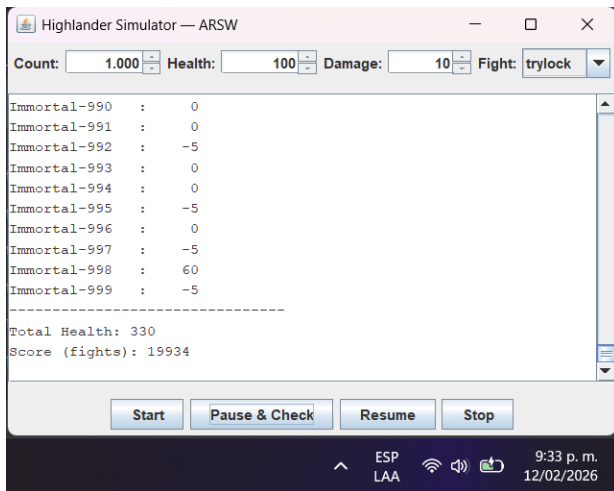


Figure 19: Validation N=1000 with trylock strategy.

8.3 N=10000 Results

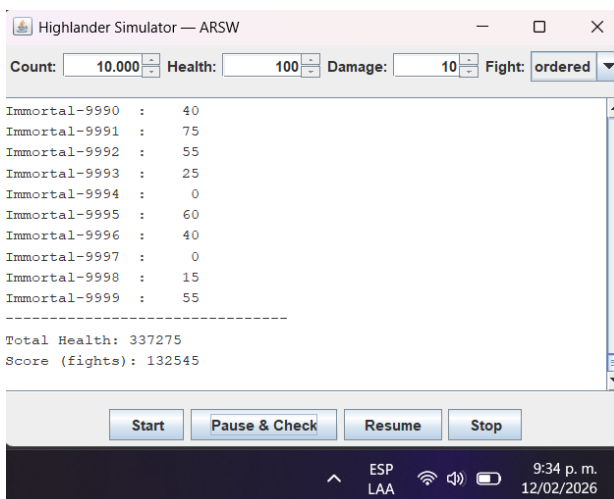


Figure 20: Validation N=10000 with ordered strategy.

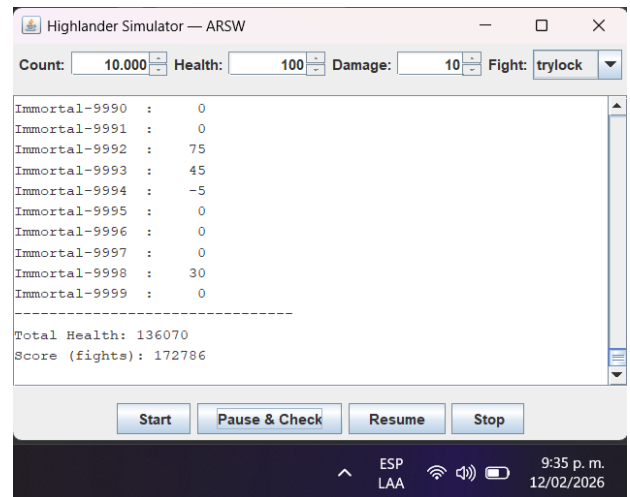


Figure 21: Validation N=10000 with trylock strategy.

8.4 Validation Summary

N	Strategy	Score	Total	Match?
100	ordered	1845	775	✓
100	trylock	1868	660	✓
1000	ordered	19487	2565	✓
1000	trylock	19934	330	✓
10000	ordered	132545	337275	✓
10000	trylock	172786	136070	✓

Table 5: High N validation results

9 Removing Dead Immortals

9.1 Problem: Race Conditions with ArrayList

The original implementation used `ArrayList<Immortal>` which causes:

- `ConcurrentModificationException` during iteration
- `IndexOutOfBoundsException` on concurrent access
- Stale size values

9.2 Solution: CopyOnWriteArrayList

`CopyOnWriteArrayList` provides thread-safe operations:

Listing 14: Lock-Free Collection

```
1 private final List<Immortal> population =
2   new CopyOnWriteArrayList<>();
```

Key properties:

- Thread-safe without explicit synchronization
- Snapshot iteration (no CME)
- Lock-free reads
- Copy-on-write semantics

Aspect	ArrayList+sync	COWAL
Reads	Blocked	Lock-free
Writes	Fast+lock	Slower (copy)
Iteration	Requires sync	Safe
Best for	Frequent writes	Frequent reads

Table 6: Collection comparison

10 Orderly Shutdown

10.1 Graceful Shutdown Pattern

The improved implementation follows:

1. Resume if paused (allow threads to check stop flag)
2. Signal all threads to stop (`running = false`)
3. Initiate graceful shutdown (`shutdown()`)
4. Wait with timeout for threads to finish
5. Force shutdown if timeout exceeded

Listing 15: Orderly Shutdown Implementation

```

1 public void stop() {
2     if (exec == null) return;
3
4     // Step 1: Resume if paused
5     if (controller.paused()) {
6         controller.resume();
7     }
8
9     // Step 2: Signal stop
10    for (Immutable im : population) {
11        im.stop();
12    }
13
14    // Step 3: Graceful shutdown
15    exec.shutdown();
16
17    try {
18        // Step 4: Wait with timeout
19        if (!exec.awaitTermination(5, SECONDS)) {
20            // Step 5: Force shutdown
21            exec.shutdownNow();
22        }
23    } catch (InterruptedException ie) {
24        exec.shutdownNow();
25        Thread.currentThread().interrupt();
26    }
27 }
```

11 Methodology

11.1 Test Environment

Component	Specification
Java Version	OpenJDK 21
Build Tool	Maven 3.9+
Test Framework	JUnit 5
Coverage Tool	JaCoCo 0.8.12
Code Analysis	SonarCloud
CI/CD	GitHub Actions

Table 7: Development environment

11.2 Execution Commands

Graphical interface (UI):

```

mvn -q -DskipTests exec:java
-Dmode=ui
-Dcount=8
-Dflight=ordered
-Dhealth=100
-Ddamage=10
```

Theoretical demos:

```
mvn -q -DskipTests exec:java
```

```

-Dmode=demos -Ddemo=1 # Naive deadlock
mvn -q -DskipTests exec:java
-Dmode=demos -Ddemo=2 # Total order
mvn -q -DskipTests exec:java
-Dmode=demos -Ddemo=3 # TryLock
```

11.3 Running Tests

```
mvn clean verify
```

12 Conclusions

Key Findings

1. **wait()/notify() for Efficiency:** The monitor pattern eliminates busy-wait, drastically reducing CPU consumption.
2. **AtomicInteger for Counters:** Lock-free counters prevent race conditions in distributed search patterns.
3. **Total Order for Deadlocks:** Establishing a global lock ordering eliminates circular wait.
4. **TryLock for Flexibility:** Timeout-based locking with exponential backoff provides adaptive contention handling.
5. **CopyOnWriteArrayList for Safe Iteration:** Lock-free reads with snapshot iteration prevent CME without global synchronization.
6. **Synchronization Barrier:** Ensures consistent state inspection during Pause & Check.

12.1 Lessons Learned

- Always use `while` loops with `wait()` for spurious wake-ups.
- Prefer `notifyAll()` over `notify()` when multiple threads may be waiting.
- Use `AtomicInteger` for simple counters instead of `synchronized`.
- Establish consistent lock ordering to prevent deadlocks.
- Use timeout-based locking when progress is more important than guaranteed execution.
- Choose concurrent collections based on read/write frequency.

13 References

1. Goetz, B. et al. (2006). *Java Concurrency in Practice*. Addison-Wesley Professional. Pages 1–4 and 15–21.
2. Oracle. Java 21 Documentation - Concurrency. <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Thread.html>
3. Lea, D. (2000). *Concurrent Programming in Java: Design Principles and Patterns*. Addison-Wesley.
4. Herlihy, M., Shavit, N. (2012). *The Art of Multiprocessor Programming*. Morgan Kaufmann.